

TRIBHUWAN UNIVERSITY

CENTRAL DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Neural Network

Assignment No : 2

Submitted By:

Name: Yujan Basnet

Sec: A

Roll No: 18

MSc CSIT

Submitted To:

Asst. Prof. Arjun Saud

CDCSIT, TU

Lab Assignment 2

**Implement Backpropagation algorithm to train an ANN of configuration 2x2x1 to achieve XOR function. (Use sigmoid and Tanh activation function).
You have to implement and online as well as batch gradient descent.**

Question 1: Implementing XOR Using Sigmoid and Online

Code:

```
# Q1
# Implement Backpropagation to train an ANN with configuration 2x2x1 for XOR function
# Using the sigmoid activation function to keep outputs in range (0,1)
# Implementing online (stochastic) learning

import numpy as np # Importing the numpy library for numerical operations

# Defining the XORNeuralNetwork class
class XORNeuralNetwork:
    def __init__(self, num_inputs=2, num_hidden=2, num_outputs=1, learning_rate=0.5,
activation_factor=1, iterations=10000):
        # Initializing the network parameters
        self.num_inputs = num_inputs # Number of input neurons (2 for XOR)
        self.num_hidden = num_hidden # Number of hidden neurons (2)
        self.num_outputs = num_outputs # Number of output neurons (1 for XOR)
        self.learning_rate = learning_rate # Learning rate for weight updates
        self.activation_factor = activation_factor # Factor to control sigmoid steepness
        self.iterations = iterations # Number of iterations for training (No of epochs)

        # Initialize weights and biases with random values
        np.random.seed(1) # Set the seed for reproducibility

        ## Initializing weights between the input layer and hidden layer
        # Shape: (num_hidden, num_inputs) -> Each hidden neuron gets weights for all input
neurons
        self.weights_input_hidden = np.random.uniform(-1, 1, (num_hidden, num_inputs))

        ## Initializing biases for the hidden layer
        # Shape: (num_hidden, 1) -> Each hidden neuron gets its own bias term
        self.bias_hidden = np.random.uniform(-1, 1, (num_hidden, 1))

        ## Initializing weights between the hidden layer and output layer
        # Shape: (num_outputs, num_hidden) -> Each output neuron gets weights from all hidden
neurons
        self.weights_hidden_output = np.random.uniform(-1, 1, (num_outputs, num_hidden))
```

```

    ## Initializing biases for the output layer
    # Shape: (num_outputs, 1) -> Each output neuron gets its own bias term
    self.bias_output = np.random.uniform(-1, 1, (num_outputs, 1))

def sigmoid(self, x):
    # Sigmoid activation function, squashes input between 0 and 1
    return 1 / (1 + np.exp(-self.activation_factor * x))

def sigmoid_derivative(self, x):
    # Derivative of the sigmoid function, needed for backpropagation
    return x * (1 - x)

def forward_propagation(self, input_vector):
    # Forward propagation to calculate the predicted output

    ## Input to hidden layer
    # Compute the weighted sum of inputs to hidden layer neurons (netinput = Wx + b)
    #  $net\_input(h) = W^{(h)} * x + b^{(h)}$ 
    hidden_net_input = np.dot(self.weights_input_hidden, input_vector) + self.bias_hidden

    ## Activation after applying sigmoid to hidden layer input
    #  $sigmoid\_activation\_func(h) = 1 / (1 + \exp(-a * net\_input(h)))$ 
    # Where 'a' is the activation factor that controls the steepness of the sigmoid function.
    hidden_activation = self.sigmoid(hidden_net_input)

    ## Forward propagation to output layer
    # Compute the weighted sum of the hidden layer activations
    # Explanation:
    # - `weights_hidden_output`: A matrix containing the weights connecting the hidden layer
    to the output layer.
    # - `hidden_activation`: The activations (output values) from the hidden layer neurons.
    output_net_input = np.dot(self.weights_hidden_output, hidden_activation) +
    self.bias_output

    # Apply sigmoid activation function to get the final predicted output
    # Explanation:
    # - `activation_factor`: A scalar value controlling how steep the activation curve is.
    predicted_output = self.sigmoid(output_net_input)
    return hidden_activation, predicted_output # Return hidden activation and predicted output

def backpropagation(self, input_vector, target_vector, hidden_activation, predicted_output):
    # Backpropagation to adjust weights and biases based on error
    output_error = target_vector - predicted_output # Error in output layer (error=y_true
    -y_pred)

    ## Gradient for output layer

```

```

# ( $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$ ) where  $\sigma(x)$  is the sigmoid activation function
output_gradient = output_error * self.sigmoid_derivative(predicted_output)

## Propagate error to hidden layer
# The error for the hidden layer is calculated by
# multiplying the output gradient by the transpose of the weights between the hidden and
output layers.
# Explanation:
# - `weights_hidden_output.T`: Displays input values in a readable row format.
hidden_error = np.dot(self.weights_hidden_output.T, output_gradient)

hidden_gradient = hidden_error * self.sigmoid_derivative(hidden_activation) # Gradient
for hidden layer

# Update weights and biases using the gradients and learning rate
self.weights_hidden_output += self.learning_rate * np.dot(output_gradient,
hidden_activation.T) # Update hidden-to-output weights
self.bias_output += self.learning_rate * output_gradient # Update output bias
self.weights_input_hidden += self.learning_rate * np.dot(hidden_gradient,
input_vector.T) # Update input-to-hidden weights
self.bias_hidden += self.learning_rate * hidden_gradient # Update hidden bias

def train(self, input_data, target_data):
    # Training the model using backpropagation
    for epoch in range(self.iterations): # Loop over the number of training iterations
        for sample in range(len(input_data)): # Loop through each training sample (XOR inputs)

            # Reshape input vector for matrix operations
            # Extract the current input sample and reshape it into a column vector (2x1)
            # Example: [0, 0] -> [[0], [0]]
            input_vector = input_data[sample].reshape(-1, 1)

            # Reshape target vector
            # Extract the corresponding target output and reshape it into a column vector (1x1)
            # Example: 0 -> [[0]]
            target_vector = np.array([target_data[sample]]).reshape(-1, 1)

            # Perform forward propagation
            hidden_activation, predicted_output = self.forward_propagation(input_vector)

            # Perform backpropagation to adjust weights
            self.backpropagation(input_vector, target_vector, hidden_activation, predicted_output)

def predict(self, input_data):
    # Making predictions with the trained model
    predictions = [] # List to store predictions

```

```

        for sample in range(len(input_data)): # Loop through each sample in the input data
            input_vector = input_data[sample].reshape(-1, 1) # Reshape input vector for matrix
operations
            _, predicted_output = self.forward_propagation(input_vector) # Perform forward
propagation to get the predicted output
            predicted_label = 1 if predicted_output > 0.5 else 0 # Convert output to binary (1 or 0)
based on threshold
            predictions.append(predicted_label) # Append prediction to list
        return predictions # Return list of predictions

# Initialize dataset for XOR function
input_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # Possible input combinations for XOR
target_data = np.array([0, 1, 1, 0]) # Target outputs for XOR

# Create the neural network model and train it
model = XORNeuralNetwork() # Initialize the XOR neural network
model.train(input_data, target_data) # Train the model with the XOR dataset

# Test the trained model by making predictions on the XOR inputs
predictions = model.predict(input_data) # Get predictions from the trained model

# enumerate function loops over a list where i -> Index and pred -> value
for i, pred in enumerate(predictions): # Loop through predictions and display them
    print(f"Input: {input_data[i]} -> Predicted: {pred} -> Target: {target_data[i]}") # Print input,
predicted, and actual target

```

Output:

```

Input: [0 0] -> Predicted: 0 -> Target: 0
Input: [0 1] -> Predicted: 1 -> Target: 1
Input: [1 0] -> Predicted: 1 -> Target: 1
Input: [1 1] -> Predicted: 0 -> Target: 0

```

Question 2: Implementing XOR using Tanh Activation function and online method

Code

```
# Q2
# Implement Backpropagation algorithm to train an ANN of configuration 2x2x1 to achieve
XOR function.
# (Use Tanh activation function)It squashes the O/P in the range of (-1,1).
# implement online method.

import numpy as np

class XORNeuralNetwork:
    def __init__(self, num_input=2, num_hidden=2, num_output=1, alpha=0.5,
num_epochs=10000):
        # Initialize network configuration and learning parameters
        self.num_input = num_input
        self.num_hidden = num_hidden
        self.num_output = num_output
        self.alpha = alpha # Learning rate
        self.num_epochs = num_epochs

        # Initialize weights and biases
        np.random.seed(1) # Ensure reproducibility
        self.weights_input_hidden = np.random.uniform(-1, 1, (self.num_hidden, self.num_input))
        self.bias_hidden = np.random.uniform(-1, 1, (self.num_hidden, 1))
        self.weights_hidden_output = np.random.uniform(-1, 1, (self.num_output,
self.num_hidden))
        self.bias_output = np.random.uniform(-1, 1, (self.num_output, 1))

    def tanh(self, x):
        return np.tanh(x)

    def tanh_derivative(self, x):
        return 1 - np.tanh(x) ** 2

    def forward_propagation(self, sample_input):
        # Forward pass to calculate the network's output
        hidden_layer_input = np.dot(self.weights_input_hidden, sample_input) + self.bias_hidden
        hidden_layer_output = self.tanh(hidden_layer_input)
        final_layer_input = np.dot(self.weights_hidden_output, hidden_layer_output) +
self.bias_output
        predicted_value = self.tanh(final_layer_input)
        return hidden_layer_input, hidden_layer_output, final_layer_input, predicted_value

    def backpropagation(self, sample_input, target_value, hidden_layer_input,
hidden_layer_output, final_layer_input, predicted_value):
        # Compute error gradients for the backpropagation step
```

```

        output_error_gradient = (target_value - predicted_value) *
self.tanh_derivative(final_layer_input)
        hidden_error_gradient = np.dot(self.weights_hidden_output.T, output_error_gradient) *
self.tanh_derivative(hidden_layer_input)

        # Update weights and biases based on the gradients
        self.weights_hidden_output += self.alpha * np.dot(output_error_gradient,
hidden_layer_output.T)
        self.bias_output += self.alpha * output_error_gradient
        self.weights_input_hidden += self.alpha * np.dot(hidden_error_gradient, sample_input.T)
        self.bias_hidden += self.alpha * hidden_error_gradient

def train(self, input_data, expected_output):
    for epoch in range(self.num_epochs):
        print(f"\nEpoch {epoch + 1}")
        for index in range(len(input_data)):
            # Prepare input and expected output
            sample_input = input_data[index].reshape(-1, 1)
            target_value = np.array([expected_output[index]]).reshape(-1, 1)

            # Forward propagation
            hidden_layer_input, hidden_layer_output, final_layer_input, predicted_value =
self.forward_propagation(sample_input)

            # Compute loss (error)
            loss = 0.5 * (target_value - predicted_value) ** 2
            print(f'Input: {sample_input.T}, Output: {predicted_value[0,0]:.4f}, Target:
{target_value[0,0]}')

            # Backpropagation to adjust weights and biases
            self.backpropagation(sample_input, target_value, hidden_layer_input,
hidden_layer_output, final_layer_input, predicted_value)

def test(self, input_data, expected_output):
    print("\nTesting the trained network:")
    for index in range(len(input_data)):
        sample_input = input_data[index].reshape(-1, 1)
        hidden_layer_input, hidden_layer_output, final_layer_input, predicted_value =
self.forward_propagation(sample_input)

        # Determine the predicted class (1 or -1) based on the output
        predicted_class = 1 if predicted_value > 0 else -1
        print(f'Input: {sample_input.T}, Output: {predicted_value[0,0]:.4f}, Predicted:
{predicted_class}, Target: {expected_output[index]}')

# XOR input and expected output

```

```
input_data = np.array([[ -1, -1], [-1, 1], [1, -1], [1, 1]])
expected_output = np.array([-1, 1, 1, -1])

# Create and train the XOR neural network
nn = XORNeuralNetwork(num_input=2, num_hidden=2, num_output=1, alpha=0.5,
num_epochs=10000)
nn.train(input_data, expected_output)

# Test the trained network
nn.test(input_data, expected_output)
```

Output:

```
Epoch 9999
Input: [[ -1 -1]], Output: -0.9949, Target: -1
Input: [[ -1  1]], Output: 0.9963, Target: 1
Input: [[  1 -1]], Output: 0.9964, Target: 1
Input: [[  1  1]], Output: -0.9949, Target: -1

Epoch 10000
Input: [[ -1 -1]], Output: -0.9949, Target: -1
Input: [[ -1  1]], Output: 0.9963, Target: 1
Input: [[  1 -1]], Output: 0.9964, Target: 1
Input: [[  1  1]], Output: -0.9949, Target: -1

Testing the trained network:
Input: [[ -1 -1]], Output: -0.9949, Predicted: -1, Target: -1
Input: [[ -1  1]], Output: 0.9963, Predicted: 1, Target: 1
Input: [[  1 -1]], Output: 0.9964, Predicted: 1, Target: 1
Input: [[  1  1]], Output: -0.9949, Predicted: -1, Target: -1
```


Question 3: XOR sigmoid and batch

Code

```
# Q3
# Implement Backpropagation algorithm to train an ANN of configuration 2x2x1 to achieve
XOR function.
# (Use sigmoid activation function)It squashes the O/P in the range of (0,1).
# implement batch gradient descent method.
```

```
import numpy as np
```

```
class XORNeuralNetwork:
```

```
    def __init__(self, input_size=2, hidden_size=2, output_size=1, learning_rate=0.5,
epochs=10000, a=1):
```

```
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.a = a # Sigmoid slope parameter
```

```
        # Initialize weights and biases
```

```
        np.random.seed(1) # Ensure reproducibility
```

```
        self.weights_input_hidden = np.random.uniform(-1, 1, (hidden_size, input_size))
```

```
        self.bias_hidden = np.random.uniform(-1, 1, (hidden_size, 1))
```

```
        self.weights_hidden_output = np.random.uniform(-1, 1, (output_size, hidden_size))
```

```
        self.bias_output = np.random.uniform(-1, 1, (output_size, 1))
```

```
    def sigmoid(self, x):
```

```
        return 1 / (1 + np.exp(-self.a * x))
```

```
    def sigmoid_derivative(self, x):
```

```
        return self.a * x * (1 - x)
```

```
    def train(self, input_data, target_data):
```

```
        for epoch in range(self.epochs):
```

```
            print(f"\nEpoch {epoch + 1}")
```

```
            # Initialize accumulators for batch gradient descent
```

```
            # Explantion:
```

```
            # `np.zeros` Creates a matrix of zeros with the same shape as
```

```
self.weights_hidden_output.
```

```
            d_weights_hidden_output = np.zeros_like(self.weights_hidden_output)
```

```
            d_bias_output = np.zeros_like(self.bias_output)
```

```
            d_weights_input_hidden = np.zeros_like(self.weights_input_hidden)
```

```
            d_bias_hidden = np.zeros_like(self.bias_hidden)
```

```

for i in range(len(input_data)):
    x = input_data[i].reshape(-1, 1)
    target = np.array([target_data[i]]).reshape(-1, 1)

    # Forward pass
    hidden_input = np.dot(self.weights_input_hidden, x) + self.bias_hidden
    hidden_output = self.sigmoid(hidden_input)
    final_input = np.dot(self.weights_hidden_output, hidden_output) + self.bias_output
    predicted_output = self.sigmoid(final_input)

    # Compute error
    print(f"Input: {x.T}, Output: {predicted_output[0,0]:.4f}, Target: {target[0,0]}")

    # Backpropagation
    output_error = (target - predicted_output) * self.sigmoid_derivative(predicted_output)
    hidden_error = np.dot(self.weights_hidden_output.T, output_error) *
self.sigmoid_derivative(hidden_output)

    # Accumulate gradients
    d_weights_hidden_output += np.dot(output_error, hidden_output.T)
    d_bias_output += output_error
    d_weights_input_hidden += np.dot(hidden_error, x.T)
    d_bias_hidden += hidden_error

    # Apply batch updates
    batch_size = len(input_data)
    self.weights_hidden_output += (self.learning_rate / batch_size) *
d_weights_hidden_output
    self.bias_output += (self.learning_rate / batch_size) * d_bias_output
    self.weights_input_hidden += (self.learning_rate / batch_size) * d_weights_input_hidden
    self.bias_hidden += (self.learning_rate / batch_size) * d_bias_hidden

def predict(self, x):
    x = x.reshape(-1, 1)
    hidden_input = np.dot(self.weights_input_hidden, x) + self.bias_hidden
    hidden_output = self.sigmoid(hidden_input)
    final_input = np.dot(self.weights_hidden_output, hidden_output) + self.bias_output
    predicted_output = self.sigmoid(final_input)
    return predicted_output

def test(self, input_data, target_data):
    print("\nTesting the trained network:")
    for i in range(len(input_data)):
        x = input_data[i].reshape(-1, 1)
        predicted_output = self.predict(x)
        predicted_class = 1 if predicted_output > 0.5 else 0

```

```
print(f'Input: {x.T}, Output: {predicted_output[0,0]:.4f}, Predicted: {predicted_class},  
Target: {target_data[i]}")
```

```
# XOR dataset
```

```
input_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

```
target_output = np.array([0, 1, 1, 0])
```

```
# Train and test the neural network
```

```
nn = XORNeuralNetwork()
```

```
nn.train(input_data, target_output)
```

```
nn.test(input_data, target_output)
```

Output

```
Epoch 9999  
Input: [[0 0]], Output: 0.0381, Target: 0  
Input: [[0 1]], Output: 0.9552, Target: 1  
Input: [[1 0]], Output: 0.9552, Target: 1  
Input: [[1 1]], Output: 0.0440, Target: 0  
  
Epoch 10000  
Input: [[0 0]], Output: 0.0381, Target: 0  
Input: [[0 1]], Output: 0.9552, Target: 1  
Input: [[1 0]], Output: 0.9552, Target: 1  
Input: [[1 1]], Output: 0.0440, Target: 0  
  
Testing the trained network:  
Input: [[0 0]], Output: 0.0380, Predicted: 0, Target: 0  
Input: [[0 1]], Output: 0.9552, Predicted: 1, Target: 1  
Input: [[1 0]], Output: 0.9552, Predicted: 1, Target: 1  
Input: [[1 1]], Output: 0.0440, Predicted: 0, Target: 0
```

Question 4: XOR using tanh and batch

Code

#Q4

Implement Backpropagation algorithm to train an ANN of configuration 2x2x1 to achieve XOR function.

(Use Tanh activation function)It squashes the O/P in the range of (-1,1).

implement batch gradient descent method.

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.5, epochs=10000):  
        np.random.seed(1) # Fix seed for reproducibility
```

```
        # Initialize weights and biases
```

```
        self.weights_input_hidden = np.random.uniform(-1, 1, (hidden_size, input_size))
```

```
        self.bias_hidden = np.random.uniform(-1, 1, (hidden_size, 1))
```

```
        self.weights_hidden_output = np.random.uniform(-1, 1, (output_size, hidden_size))
```

```
        self.bias_output = np.random.uniform(-1, 1, (output_size, 1))
```

```
        self.learning_rate = learning_rate
```

```
        self.epochs = epochs
```

```
    def tanh(self, x):
```

```
        return np.tanh(x)
```

```
    def tanh_derivative(self, x):
```

```
        return 1 - np.tanh(x) ** 2 # Derivative of tanh
```

```
    def forward(self, x):
```

```
        """ Forward propagation """
```

```
        self.hidden_input = np.dot(self.weights_input_hidden, x) + self.bias_hidden
```

```
        self.hidden_output = self.tanh(self.hidden_input)
```

```
        self.output_input = np.dot(self.weights_hidden_output, self.hidden_output) +
```

```
        self.bias_output
```

```
        self.output = self.tanh(self.output_input)
```

```
        return self.output
```

```
    def backward(self, inputs, targets):
```

```
        """ Batch Gradient Descent - Computes gradients over entire dataset """
```

```
        d_weights_hidden_output = np.zeros_like(self.weights_hidden_output)
```

```
        d_bias_output = np.zeros_like(self.bias_output)
```

```
        d_weights_input_hidden = np.zeros_like(self.weights_input_hidden)
```

```
        d_bias_hidden = np.zeros_like(self.bias_hidden)
```

```

batch_size = len(inputs)

for i in range(batch_size):
    x = inputs[i].reshape(-1, 1)
    target = np.array([targets[i]]).reshape(-1, 1)

    # Forward pass
    self.forward(x)

    # Compute output error gradient
    output_error = (target - self.output) * self.tanh_derivative(self.output_input)

    # Compute hidden layer error gradient
    hidden_error = np.dot(self.weights_hidden_output.T, output_error) *
self.tanh_derivative(self.hidden_input)

    # Accumulate gradients
    d_weights_hidden_output += np.dot(output_error, self.hidden_output.T)
    d_bias_output += output_error
    d_weights_input_hidden += np.dot(hidden_error, x.T)
    d_bias_hidden += hidden_error

    # Update weights and biases with batch gradient descent
    self.weights_hidden_output += (self.learning_rate / batch_size) * d_weights_hidden_output
    self.bias_output += (self.learning_rate / batch_size) * d_bias_output
    self.weights_input_hidden += (self.learning_rate / batch_size) * d_weights_input_hidden
    self.bias_hidden += (self.learning_rate / batch_size) * d_bias_hidden

def train(self, inputs, targets):
    """ Train the neural network using batch gradient descent """
    for epoch in range(self.epochs):
        self.backward(inputs, targets)

def predict(self, x):
    """ Predict output for given input """
    x = x.reshape(-1, 1)
    output = self.forward(x)
    return 1 if output > 0 else -1 # Convert tanh output to binary (-1 or 1)

# Initialize XOR dataset
XOR_inputs = np.array([[-1, -1], [-1, 1], [1, -1], [1, 1]])
XOR_targets = np.array([-1, 1, 1, -1])

# Create and train the neural network
nn = NeuralNetwork(input_size=2, hidden_size=2, output_size=1)
nn.train(XOR_inputs, XOR_targets)

```

```
# Test the trained network
print("\nTesting the trained network:")
for i in range(len(XOR_inputs)):
    prediction = nn.predict(XOR_inputs[i])
    print(f"Input: {XOR_inputs[i]}, Predicted: {prediction}, Target: {XOR_targets[i]}")
```

Output

```
Testing the trained network:
Input: [-1 -1], Predicted: -1, Target: -1
Input: [-1  1], Predicted: 1, Target: 1
Input: [ 1 -1], Predicted: 1, Target: 1
Input: [1  1], Predicted: -1, Target: -1
```

Implement Backpropagation algorithm to train an ANN of configuration 3x2x2x1 to achieve majority function with 3-bit data. Output of the network must be 1 when there are two or more 1's in the data. (Use sigmoid and Tanh activation function). You have to implement and online as well as batch gradient descent.

Question 5: Using Sigmoid and Online

Code

Q5

#Implement Backpropagation algorithm to train an ANN of configuration 3x2x2x1 to achieve majority function with 3-bit data.

#Output of the network must be 1 when there are two or more 1's in the data.

#(Use sigmoid activation function).

#implementing and online method.

import numpy as np

class NeuralNetwork:

def __init__(self, input_size, hidden_size1, hidden_size2, output_size, learning_rate=0.5):

"""

Initializes the neural network with random weights and biases.

:param input_size: Number of neurons in the input layer

:param hidden_size1: Number of neurons in the first hidden layer

:param hidden_size2: Number of neurons in the second hidden layer

:param output_size: Number of neurons in the output layer

:param learning_rate: Learning rate for weight updates

"""

self.input_size = input_size

self.hidden_size1 = hidden_size1

self.hidden_size2 = hidden_size2

self.output_size = output_size

self.learning_rate = learning_rate

self.a = 1 # Sigmoid activation parameter

Set a random seed for reproducibility

np.random.seed(1)

Initialize weights and biases for each layer

self.W1 = np.random.uniform(-1, 1, (hidden_size1, input_size)) # Weights between input
and hidden layer 1

self.b1 = np.random.uniform(-1, 1, (hidden_size1, 1)) # Bias for hidden layer 1

self.W2 = np.random.uniform(-1, 1, (hidden_size2, hidden_size1)) # Weights between
hidden layer 1 and 2

self.b2 = np.random.uniform(-1, 1, (hidden_size2, 1)) # Bias for hidden layer 2

```

        self.W3 = np.random.uniform(-1, 1, (output_size, hidden_size2)) # Weights between
hidden layer 2 and output
        self.b3 = np.random.uniform(-1, 1, (output_size, 1)) # Bias for output layer

def sigmoid(self, x):
    """Applies the sigmoid activation function."""
    return 1 / (1 + np.exp(-self.a * x))

def sigmoid_derivative(self, x):
    """Computes the derivative of the sigmoid function."""
    return self.a * x * (1 - x)

def forward(self, x):
    """Performs the forward propagation through the network."""
    self.hidden_layer1 = np.dot(self.W1, x) + self.b1
    self.a1 = self.sigmoid(self.hidden_layer1) # Activation of first hidden layer

    self.hidden_layer2 = np.dot(self.W2, self.a1) + self.b2
    self.a2 = self.sigmoid(self.hidden_layer2) # Activation of second hidden layer

    self.output_layer = np.dot(self.W3, self.a2) + self.b3
    self.output = self.sigmoid(self.output_layer) # Final output
    return self.output

def backward(self, x, target):
    """Performs the backward propagation (Backpropagation algorithm)."""
    error = target - self.output # Compute error
    d_output = error * self.sigmoid_derivative(self.output) # Output layer error gradient

    d_hidden2 = np.dot(self.W3.T, d_output) * self.sigmoid_derivative(self.a2) # Hidden layer
2 error gradient
    d_hidden1 = np.dot(self.W2.T, d_hidden2) * self.sigmoid_derivative(self.a1) # Hidden
layer 1 error gradient

    # Update weights and biases
    self.W3 += self.learning_rate * np.dot(d_output, self.a2.T)
    self.b3 += self.learning_rate * d_output
    self.W2 += self.learning_rate * np.dot(d_hidden2, self.a1.T)
    self.b2 += self.learning_rate * d_hidden2
    self.W1 += self.learning_rate * np.dot(d_hidden1, x.T)
    self.b1 += self.learning_rate * d_hidden1

def train(self, X, Y, epochs):
    """Trains the neural network using the given dataset."""
    for epoch in range(epochs):
        # print(f"\nEpoch {epoch + 1}")

```



```

    for i in range(len(X)):
        x = X[i].reshape(-1, 1) # Reshape input as column vector
        target = np.array([Y[i]]).reshape(-1, 1) # Reshape target as column vector
        self.forward(x) # Forward propagation
        self.backward(x, target) # Backpropagation and weight update
        # print(f'Input: {x.T}, Output: {self.output[0,0]:.4f}, Target: {target[0,0]}")

def predict(self, X):
    """Tests the trained model with the given inputs."""
    print("\nTesting the trained network:")
    for i in range(len(X)):
        x = X[i].reshape(-1, 1)
        output = self.forward(x) # Perform forward pass
        predicted_class = 1 if output > 0.5 else 0 # Round output to 0 or 1
        print(f'Input: {x.T}, Output: {output[0,0]:.4f}, Predicted: {predicted_class}")

# Define input and target data for majority function (3-bit input)
input_values = np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1],
                          [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
target_outputs = np.array([0, 0, 0, 1, 0, 0, 1, 1]) # Output is 1 when at least 2 bits are 1

# Create and train the neural network
nn = NeuralNetwork(input_size=3, hidden_size1=2, hidden_size2=2, output_size=1,
                    learning_rate=0.5)
nn.train(input_values, target_outputs, epochs=10000)

# Test the trained model
nn.predict(input_values)

```

Output

```

Testing the trained network:
Input: [[0 0 0]], Output: 0.0019, Predicted: 0
Input: [[0 0 1]], Output: 0.0066, Predicted: 0
Input: [[0 1 0]], Output: 0.0060, Predicted: 0
Input: [[0 1 1]], Output: 0.9936, Predicted: 1
Input: [[1 0 0]], Output: 0.0064, Predicted: 0
Input: [[1 0 1]], Output: 0.9936, Predicted: 1
Input: [[1 1 0]], Output: 0.9933, Predicted: 1
Input: [[1 1 1]], Output: 0.9975, Predicted: 1

```

Question 6: sigmoid and batch

Code

#Q6

#Implement Backpropagation algorithm to train an ANN of configuration 3x2x2x1 to achieve majority function with 3-bit data.

#Output of the network must be 1 when there are two or more 1's in the data.

#(Use sigmoid activation function).

#implementing batch gradient descent method.

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size):
```

```
        np.random.seed(1) # Fix random seed for reproducibility
```

```
        # Initialize layers and parameters
```

```
        self.input_size = input_size
```

```
        self.hidden_size1 = hidden_size1
```

```
        self.hidden_size2 = hidden_size2
```

```
        self.output_size = output_size
```

```
        # Initialize weights and biases
```

```
        self.weights_input_hidden1 = np.random.uniform(-1, 1, (hidden_size1, input_size))
```

```
        self.bias_hidden1 = np.random.uniform(-1, 1, (hidden_size1, 1))
```

```
        self.weights_hidden1_hidden2 = np.random.uniform(-1, 1, (hidden_size2, hidden_size1))
```

```
        self.bias_hidden2 = np.random.uniform(-1, 1, (hidden_size2, 1))
```

```
        self.weights_hidden2_output = np.random.uniform(-1, 1, (output_size, hidden_size2))
```

```
        self.bias_output = np.random.uniform(-1, 1, (output_size, 1))
```

```
        self.learning_rate = 0.5
```

```
        self.a = 1 # Sigmoid slope parameter
```

```
    def sigmoid(self, x):
```

```
        """Sigmoid activation function."""
```

```
        return 1 / (1 + np.exp(-self.a * x))
```

```
    def forward_pass(self, x):
```

```
        """Forward pass through the network."""
```

```
        self.hidden1_input = np.dot(self.weights_input_hidden1, x) + self.bias_hidden1
```

```
        self.hidden1_output = self.sigmoid(self.hidden1_input)
```

```
        self.hidden2_input = np.dot(self.weights_hidden1_hidden2, self.hidden1_output) +  
self.bias_hidden2
```

```
        self.hidden2_output = self.sigmoid(self.hidden2_input)
```

```

        self.output_input = np.dot(self.weights_hidden2_output, self.hidden2_output) +
self.bias_output
        self.output = self.sigmoid(self.output_input)

    return self.output

def backward_pass(self, x, target):
    """Backward pass (backpropagation) for weight updates."""
    # Calculate error gradients for output layer
    output_error_gradient = (target - self.output) * (self.a * self.output * (1 - self.output))

    # Calculate error gradients for hidden layer 2
    hidden2_error_gradient = np.dot(self.weights_hidden2_output.T, output_error_gradient) *
(self.a * self.hidden2_output * (1 - self.hidden2_output))

    # Calculate error gradients for hidden layer 1
    hidden1_error_gradient = np.dot(self.weights_hidden1_hidden2.T, hidden2_error_gradient)
* (self.a * self.hidden1_output * (1 - self.hidden1_output))

    # Accumulate gradients
    self.d_weights_hidden2_output += np.dot(output_error_gradient, self.hidden2_output.T)
    self.d_bias_output += output_error_gradient

    self.d_weights_hidden1_hidden2 += np.dot(hidden2_error_gradient, self.hidden1_output.T)
    self.d_bias_hidden2 += hidden2_error_gradient

    self.d_weights_input_hidden1 += np.dot(hidden1_error_gradient, x.T)
    self.d_bias_hidden1 += hidden1_error_gradient

def update_weights(self, batch_size):
    """Update weights and biases using the accumulated gradients."""
    self.weights_hidden2_output += (self.learning_rate / batch_size) *
self.d_weights_hidden2_output
    self.bias_output += (self.learning_rate / batch_size) * self.d_bias_output

    self.weights_hidden1_hidden2 += (self.learning_rate / batch_size) *
self.d_weights_hidden1_hidden2
    self.bias_hidden2 += (self.learning_rate / batch_size) * self.d_bias_hidden2

    self.weights_input_hidden1 += (self.learning_rate / batch_size) *
self.d_weights_input_hidden1
    self.bias_hidden1 += (self.learning_rate / batch_size) * self.d_bias_hidden1

def train(self, inputs, targets, epochs):
    """Train the network using batch gradient descent."""

```

```

for epoch in range(epochs):
    # print(f"\nEpoch {epoch + 1}")

    # Reset gradients
    self.d_weights_hidden2_output = np.zeros_like(self.weights_hidden2_output)
    self.d_bias_output = np.zeros_like(self.bias_output)
    self.d_weights_hidden1_hidden2 = np.zeros_like(self.weights_hidden1_hidden2)
    self.d_bias_hidden2 = np.zeros_like(self.bias_hidden2)
    self.d_weights_input_hidden1 = np.zeros_like(self.weights_input_hidden1)
    self.d_bias_hidden1 = np.zeros_like(self.bias_hidden1)

    # Iterate over all training samples (batch gradient descent)
    for i in range(len(inputs)):
        x = inputs[i].reshape(-1, 1) # Reshape input to column vector
        target = np.array([targets[i]]).reshape(-1, 1) # Reshape target to column vector

        # Forward pass
        self.forward_pass(x)

        # Backward pass
        self.backward_pass(x, target)

    # Update weights and biases
    self.update_weights(len(inputs))

def predict(self, inputs):
    """Predict the output for new inputs."""
    predictions = []
    for x in inputs:
        x = x.reshape(-1, 1)
        output = self.forward_pass(x)
        predictions.append(1 if output > 0.5 else 0) # Convert to binary (0 or 1)
    return predictions

# XOR Input values and targets
input_values = np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
target_outputs = np.array([0, 0, 0, 1, 0, 1, 1, 1])

# Initialize the neural network
nn = NeuralNetwork(input_size=3, hidden_size1=2, hidden_size2=2, output_size=1)

# Train the network
nn.train(input_values, target_outputs, epochs=10000)

# After training, test the network

```

```
print("\nTesting the trained network:")
predictions = nn.predict(input_values)

# zip(input_values, predictions, target_outputs) combines the three lists into tuples, where each
# tuple contains one element from each list.
# enumerate() adds an index i to each tuple.
for i, (input_data, prediction, target) in enumerate(zip(input_values, predictions, target_outputs)):
    print(f"Input: {input_data}, Predicted: {prediction}, Target: {target}")
```

Output

```
Testing the trained network:
Input: [0 0 0], Predicted: 0, Target: 0
Input: [0 0 1], Predicted: 0, Target: 0
Input: [0 1 0], Predicted: 0, Target: 0
Input: [0 1 1], Predicted: 1, Target: 1
Input: [1 0 0], Predicted: 0, Target: 0
Input: [1 0 1], Predicted: 1, Target: 1
Input: [1 1 0], Predicted: 1, Target: 1
Input: [1 1 1], Predicted: 1, Target: 1
```

Question 7: tanh and batch Code

Q7

#Implement Backpropagation algorithm to train an ANN of configuration 3x2x2x1 to achieve majority function with 3-bit data.

#Output of the network must be 1 when there are two or more 1's in the data.

#(Use tanh activation function).

#implementing and online method.

import numpy as np

class NeuralNetwork:

def __init__(self, input_size, hidden1_size, hidden2_size, output_size, learning_rate=0.5, epochs=10000):

np.random.seed(1) # Fix random seed for reproducibility

self.input_size = input_size

self.hidden1_size = hidden1_size

self.hidden2_size = hidden2_size

self.output_size = output_size

self.learning_rate = learning_rate

self.epochs = epochs

Initialize weights and biases

self.W1 = np.random.uniform(-1, 1, (hidden1_size, input_size))

self.b1 = np.random.uniform(-1, 1, (hidden1_size, 1))

self.W2 = np.random.uniform(-1, 1, (hidden2_size, hidden1_size))

self.b2 = np.random.uniform(-1, 1, (hidden2_size, 1))

self.W3 = np.random.uniform(-1, 1, (output_size, hidden2_size))

self.b3 = np.random.uniform(-1, 1, (output_size, 1))

def tanh(self, x):

return np.tanh(x)

def tanh_derivative(self, x):

return 1 - np.tanh(x) ** 2

def forward(self, x):

self.z1 = np.dot(self.W1, x) + self.b1

self.a1 = self.tanh(self.z1)

self.z2 = np.dot(self.W2, self.a1) + self.b2

self.a2 = self.tanh(self.z2)

self.z3 = np.dot(self.W3, self.a2) + self.b3

self.output = self.tanh(self.z3)

return self.output

```

def backward(self, x, target):
    # Compute output layer error
    output_error = (target - self.output) * self.tanh_derivative(self.output)

    # Compute hidden layer errors
    hidden2_error = np.dot(self.W3.T, output_error) * self.tanh_derivative(self.a2)
    hidden1_error = np.dot(self.W2.T, hidden2_error) * self.tanh_derivative(self.a1)

    # Update weights and biases
    self.W3 += self.learning_rate * np.dot(output_error, self.a2.T)
    self.b3 += self.learning_rate * output_error

    self.W2 += self.learning_rate * np.dot(hidden2_error, self.a1.T)
    self.b2 += self.learning_rate * hidden2_error

    self.W1 += self.learning_rate * np.dot(hidden1_error, x.T)
    self.b1 += self.learning_rate * hidden1_error

def train(self, input_values, target_outputs):
    for epoch in range(self.epochs):
        for i in range(len(input_values)):
            x = input_values[i].reshape(-1, 1)
            target = np.array([target_outputs[i]]).reshape(-1, 1)

            self.forward(x)
            self.backward(x, target)

        # if epoch % 1000 == 0:
        #     print(f"Epoch {epoch+1}/{self.epochs}")

def test(self, input_values, target_outputs):
    print("\nTesting the trained network:")
    for i in range(len(input_values)):
        x = input_values[i].reshape(-1, 1)
        predicted_output = self.forward(x)
        predicted_class = 1 if predicted_output > 0.5 else 0
        print(f"Input: {x.T}, Output: {predicted_output[0,0]:.4f}, Predicted: {predicted_class},
Target: {target_outputs[i]}")

# Define input and target values
input_values = np.array([
    [0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1],
    [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]
])
target_outputs = np.array([0, 0, 0, 1, 0, 1, 1, 1])

```

```
# Initialize and train the network
nn = NeuralNetwork(input_size=3, hidden1_size=2, hidden2_size=2, output_size=1)
nn.train(input_values, target_outputs)

# Test the trained network
nn.test(input_values, target_outputs)
```

Output

```
Testing the trained network:
Input: [[0 0 0]], Output: -0.0024, Predicted: 0, Target: 0
Input: [[0 0 1]], Output: 0.0016, Predicted: 0, Target: 0
Input: [[0 1 0]], Output: -0.0027, Predicted: 0, Target: 0
Input: [[0 1 1]], Output: 1.0000, Predicted: 1, Target: 1
Input: [[1 0 0]], Output: -0.0004, Predicted: 0, Target: 0
Input: [[1 0 1]], Output: 1.0000, Predicted: 1, Target: 1
Input: [[1 1 0]], Output: 0.9999, Predicted: 1, Target: 1
Input: [[1 1 1]], Output: 1.0000, Predicted: 1, Target: 1
```


Question 8: tanh and batch

Code

#Q8

#Implement Backpropagation algorithm to train an ANN of configuration 3x2x2x1 to achieve majority function with 3-bit data.

#Output of the network must be 1 when there are two or more 1's in the data.

#(Use tanh activation function).

#implementing batch gradient descent method.

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size, learning_rate=0.5):
```

```
        np.random.seed(1) # Ensure reproducibility
```

```
        self.learning_rate = learning_rate
```

```
        # Initialize weights and biases
```

```
        self.weights_input_hidden1 = np.random.uniform(-1, 1, (hidden_size1, input_size))
```

```
        self.bias_hidden1 = np.random.uniform(-1, 1, (hidden_size1, 1))
```

```
        self.weights_hidden1_hidden2 = np.random.uniform(-1, 1, (hidden_size2, hidden_size1))
```

```
        self.bias_hidden2 = np.random.uniform(-1, 1, (hidden_size2, 1))
```

```
        self.weights_hidden2_output = np.random.uniform(-1, 1, (output_size, hidden_size2))
```

```
        self.bias_output = np.random.uniform(-1, 1, (output_size, 1))
```

```
    def tanh(self, x):
```

```
        return np.tanh(x)
```

```
    def tanh_derivative(self, x):
```

```
        return 1 - np.tanh(x) ** 2
```

```
    def forward_pass(self, x):
```

```
        """Performs a forward pass through the network."""
```

```
        self.hidden1_input = np.dot(self.weights_input_hidden1, x) + self.bias_hidden1
```

```
        self.hidden1_output = self.tanh(self.hidden1_input)
```

```
        self.hidden2_input = np.dot(self.weights_hidden1_hidden2, self.hidden1_output) +  
self.bias_hidden2
```

```
        self.hidden2_output = self.tanh(self.hidden2_input)
```

```
        self.output_input = np.dot(self.weights_hidden2_output, self.hidden2_output) +  
self.bias_output
```

```
        self.predicted_output = self.tanh(self.output_input)
```

```
        return self.predicted_output
```

```

def backward_pass(self, x, target):
    """Performs backpropagation to compute gradients."""
    # Compute error at the output layer
    output_error = (target - self.predicted_output)
    output_gradient = output_error * self.tanh_derivative(self.output_input)

    # Compute error at the second hidden layer
    hidden2_error = np.dot(self.weights_hidden2_output.T, output_gradient)
    hidden2_gradient = hidden2_error * self.tanh_derivative(self.hidden2_input)

    # Compute error at the first hidden layer
    hidden1_error = np.dot(self.weights_hidden1_hidden2.T, hidden2_gradient)
    hidden1_gradient = hidden1_error * self.tanh_derivative(self.hidden1_input)

    return output_gradient, hidden2_gradient, hidden1_gradient

def update_weights(self, x, output_gradient, hidden2_gradient, hidden1_gradient, batch_size):
    """Updates weights and biases using accumulated gradients."""
    self.weights_hidden2_output += (self.learning_rate / batch_size) * np.dot(output_gradient,
self.hidden2_output.T)
    self.bias_output += (self.learning_rate / batch_size) * output_gradient

    self.weights_hidden1_hidden2 += (self.learning_rate / batch_size) *
np.dot(hidden2_gradient, self.hidden1_output.T)
    self.bias_hidden2 += (self.learning_rate / batch_size) * hidden2_gradient

    self.weights_input_hidden1 += (self.learning_rate / batch_size) * np.dot(hidden1_gradient,
x.T)
    self.bias_hidden1 += (self.learning_rate / batch_size) * hidden1_gradient

def train(self, input_values, target_outputs, epochs=10000):
    """Trains the network using batch gradient descent."""
    for epoch in range(epochs):
        d_weights_hidden2_output = np.zeros_like(self.weights_hidden2_output)
        d_bias_output = np.zeros_like(self.bias_output)
        d_weights_hidden1_hidden2 = np.zeros_like(self.weights_hidden1_hidden2)
        d_bias_hidden2 = np.zeros_like(self.bias_hidden2)
        d_weights_input_hidden1 = np.zeros_like(self.weights_input_hidden1)
        d_bias_hidden1 = np.zeros_like(self.bias_hidden1)

        for i in range(len(input_values)):
            x = input_values[i].reshape(-1, 1)
            target = np.array([target_outputs[i]]).reshape(-1, 1)

            self.forward_pass(x)

```

```

output_gradient, hidden2_gradient, hidden1_gradient = self.backward_pass(x, target)

# Accumulate gradients
d_weights_hidden2_output += np.dot(output_gradient, self.hidden2_output.T)
d_bias_output += output_gradient
d_weights_hidden1_hidden2 += np.dot(hidden2_gradient, self.hidden1_output.T)
d_bias_hidden2 += hidden2_gradient
d_weights_input_hidden1 += np.dot(hidden1_gradient, x.T)
d_bias_hidden1 += hidden1_gradient

# Update weights after processing all samples
batch_size = len(input_values)
self.update_weights(x, output_gradient, hidden2_gradient, hidden1_gradient, batch_size)

if epoch % 1000 == 0:
    print(f"Epoch {epoch}")

def test(self, input_values, target_outputs):
    """Tests the trained network."""
    print("\nTesting the trained network:")
    for i in range(len(input_values)):
        x = input_values[i].reshape(-1, 1)
        predicted_output = self.forward_pass(x)
        predicted_class = 1 if predicted_output > 0.5 else 0
        print(f"Input: {x.T}, Output: {predicted_output[0,0]:.4f}, Predicted: {predicted_class},
Target: {target_outputs[i]}")

# Define input and target output
data_inputs = np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1,
1]])
target_outputs = np.array([0, 0, 0, 1, 0, 1, 1, 1])

# Initialize and train the neural network
nn = NeuralNetwork(input_size=3, hidden_size1=2, hidden_size2=2, output_size=1)
nn.train(data_inputs, target_outputs)
nn.test(data_inputs, target_outputs)

```

Output

```
Epoch 0
Epoch 1000
Epoch 2000
Epoch 3000
Epoch 4000
Epoch 5000
Epoch 6000
Epoch 7000
Epoch 8000
Epoch 9000

Testing the trained network:
Input: [[0 0 0]], Output: 0.9891, Predicted: 1, Target: 0
Input: [[0 0 1]], Output: 0.9915, Predicted: 1, Target: 0
Input: [[0 1 0]], Output: 0.9892, Predicted: 1, Target: 0
Input: [[0 1 1]], Output: 0.9915, Predicted: 1, Target: 1
Input: [[1 0 0]], Output: 0.9904, Predicted: 1, Target: 0
Input: [[1 0 1]], Output: 0.9917, Predicted: 1, Target: 1
Input: [[1 1 0]], Output: 0.9901, Predicted: 1, Target: 1
Input: [[1 1 1]], Output: 0.9917, Predicted: 1, Target: 1
```